Preview: Abstract, Problem, and Benchmark Fragments

pipe_design tech note

Internal preview only — not a submission. Citations marked TODO_* are placeholders.

Abstract

Additive manufacturing (AM) enables compact hydraulic components whose internal fluid channels can follow free-form 3D paths rather than conventionally drilled holes. High-performance rotary direct-drive servo valves are a representative case: a motor-driven center axis must be connected to multiple groups of external ports by short, smooth pipes that never collide with each other or with structural obstacles. Once a channel is placed it permanently occupies volume, so later channels must clear earlier geometry—unlike classical multi-agent pathfinding (MAPF), where agents free space after moving. We study this setting as *multi-group pipe routing under permanent geometric occupancy*. We formalize cross-group clearance conflicts (dual witness points with radii), describe a layered planner that adapts CBS, PBS, and priority planning to geometric constraints and multi-terminal topology, and introduce a constructive 3D benchmark with two corridor generators and two obstacle painters. On this suite we evaluate solvers by success rate under a fixed time budget. The goal of this work is a reproducible problem definition, benchmark, and baseline comparison for AM-oriented multi-group channel design.

1 Introduction and Motivation

Conventional hydraulic valves often realize internal flow paths by intersecting drilled holes. Additive manufacturing removes much of that machining constraint: channels can be shorter, smoother, and spatially interleaved inside a dense envelope, which is attractive for reducing pressure loss and package size [1].

A motivating device is the *high-performance rotary direct-drive servo valve*. Structurally it combines:

- a **center axis core** driven by a motor (rotary spool / sleeve interface);
- several **groups of external terminals** (ports) on the valve body;
- **non-conflicting fluid pipes** that connect terminals of each group toward the center axis (and related internal openings).

At different rotation angles, different groups are brought into hydraulic connection through the rotary core. Therefore pipes belonging to *different* groups must remain geometrically separated for all design poses of interest, while pipes *within* a group may share trunks or branch (tree-like manifolds), as illustrated in Fig. 1.

AM makes such layouts manufacturable, but *designing* the channels is still a combinatorial 3D routing problem: many terminals, cluttered obstacles, and hard clearance between groups. Manual CAD routing does not scale; we seek automatic planners that return conflict-free multi-group layouts inside a bounded design domain.

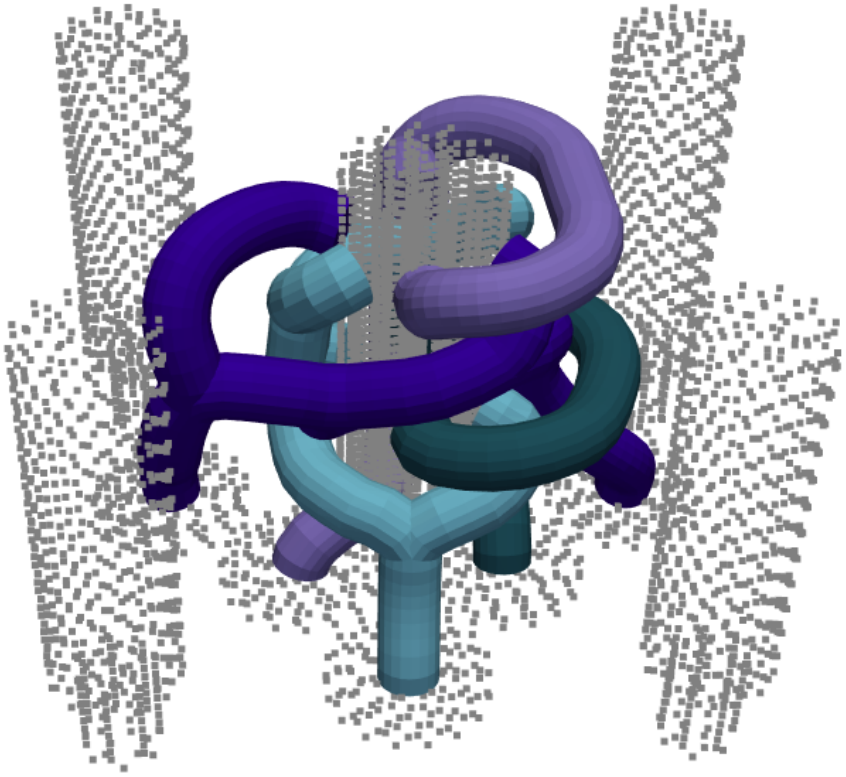


Figure 1: Illustrative multi-group fluid channels around a center axis and structural pillars (visualisation of an AM-oriented valve-style layout). Coloured tubes are pipes of different groups; grey volumes mark occupied / forbidden structure. Channels are short and smoothly curved relative to drilled manifolds, but must not collide.

2 From Valve Design to a Computational Problem

We abstract the valve setting as *multi-group pipe routing under permanent geometric occupancy*. The design domain is a bounded 3D region with static obstacles (housing, fasteners, sealed volumes, axis envelope, ...). Terminals are partitioned into groups that may share a manifold toward the axis; each routed pipe permanently occupies a tubular neighbourhood of radius r (there is no "pass and release" as in robot MAPF); cross-group geometric interference is forbidden, while within-group sharing / branching is allowed via a topology plan. This combination is not captured by classical MAPF suites, so we formalize the problem below and construct a synthetic benchmark (Sec. 3).

Geometry and occupancy. The domain is discretised as a regular 3D grid of resolution Δ (we use Δ=1 without loss of generality by scaling). A pipe of radius r occupies a *capsule* neighbourhood along its centreline (sphere swept along each segment). Static obstacles are modelled as spheres, capsules, and/or axis-aligned boxes. A query point (or cell centre) p with pipe radius r is **free** iff its clearance to all static obstacles and to already committed pipes of *other* groups is at least r. Equivalently, the open ball $B(p, r)$ must not intersect forbidden geometry.

Groups, terminals, and topology. An instance specifies m groups $G_1, \ldots, G_m$. Group G_i has terminals $T_i = \{t_{i,1}, \ldots, t_{i,n_i}\}$ on the domain boundary or on prescribed interfaces such as the axis sleeve (ports in the valve metaphor; center-axis openings may be additional terminals or topology goals). Before geometric search, a topology builder maps T_i to tasks (edges) E_i—typically a spanning tree—and an optional intra-group growth order. Solving all tasks of G_i yields a connected tree-shaped network. Cross-group tasks interact only through occupancy.

Conflicts. Unlike classical MAPF vertex/edge conflicts at a shared time, two pipes of different groups conflict when their tubular neighbourhoods violate clearance. A detected conflict stores witness points (p_a, r_a) and (p_b, r_b) (generally $p_a \neq p_b$) and a deficit measuring penetration. Same-group contacts are not conflicts under the default sharing policy.

Feasible solution and objectives. A feasible solution is a set of paths for all tasks of all groups such that every path is free w.r.t. static obstacles and all other-group committed geometry. The primary experimental objective in this paper is *feasibility under a wall-clock budget* (success rate). Secondary signals include total path cost relative to a constructive witness. Manufacturing-oriented criteria (bend count, supportability, pressure-loss proxies) matter for valves but are left to future work; we focus on the geometric multi-group core first.

Table 1: Classical MAPF vs. multi-group pipe routing under permanent occupancy.

Aspect	Classical MAPF	This problem
Occupancy	Temporary (agents move)	Permanent (pipes stay)
Conflict	Same cell / edge @ time t	Geometric clearance (dual witnesses)
Agent	Single s–t path	Multi-terminal **group** (tree)
Time	Explicit time dimension	Design-time layout (no schedule)

Relation to MAPF. As shown in Table 1, the settings differ in occupancy, conflict semantics, and agent structure. High-level MAPF controllers (CBS, PBS, priority planning) remain useful as search over leaves (constraint sets or orders), provided low-level search and conflict detection use permanent geometric occupancy. We do not claim a drop-in replacement for spatiotemporal MAPF benchmarks.

3 Benchmark Construction

Classical MAPF suites do not capture permanent geometric occupancy or multi-terminal pipe groups, and we are not aware of a public 3D multi-group pipe-routing suite with controlled difficulty and a constructive feasibility certificate. We therefore build instances synthetically.

Each instance is a triple $(\mathcal{E}, \{G_i\}_{i=1}^{m}, \mathcal{W})$: environment $\mathcal{E}$, m groups with face terminals and uniform radius r, and a conflict-free **witness** $\mathcal{W}$ proving generator-level feasibility. Solvers never see $\mathcal{W}$ as a hint; it is used for validation and as a *reference length*, not an optimality bound. Growing $\mathcal{W}$ first, then painting obstacles in its complement, yields known-feasible clutter without coupling the suite to any solver under test.

Table 2: Difficulty schedule. Shared: domain 32^3, r=0.5.

Level	AxisRay ($m \times n_t$)	PlaneSlice ($m \times n_t$)
easy	3×3	3×3
middle	4×4	6×6
hard	5×5	10×10

Pipeline. (1) Grow tree-like witness corridors with AxisRay or PlaneSlice (groups attach in round-robin). (2) Paint static obstacles outside thickened witnesses with PillarField or CorridorRack, never invading clearance r on $\mathcal{W}$. (3) Emit YAML (grid, obstacles, terminals, witness). The product of two corridor methods and two obstacle families gives **four datasets**, each with levels `easy`/`middle`/`hard` (Table 2; N=100 successes per cell, domain 32^3, r=0.5).

Corridor generators. Both produce multi-terminal trees, not a single s–t path; inter-group corridors are obstacles for later growth. AxisRay (preferred) grows an axis-aligned polyline tree by LOS scans (no subdomain grid, no A*), with path/branch gaps and forced axis changes at turns. PlaneSlice grows on axis-aligned planes with *planar* A* only (no 3D fallback), using an $N \times N \times N$ subdomain layout and a plane-gap within each group. Pseudocode and clearance rules appear in Appendix A.

Obstacle generators. Witness cells are thickened into a keep-out set C; obstacles never invade C. PillarField places large AABBs until target density $\rho^\star = V_{\text{obs}}/V_0$ (we use $\rho^\star$=0.5), sampling seeds by distance to blocked cells. CorridorRack dilates witnesses by k Chebyshev steps and meshes the complement as AABBs (density derived). Details in Appendix A.

4 Evaluation Protocol

Given $(\mathcal{E}, \{G_i\})$ without $\mathcal{W}$, a solver must return conflict-free paths for all topology tasks or fail within budget. Reported runs fix topology to pure MST and low-level search to A* (26-connected, Manhattan), and compare CBS / PBS / PP.

Primary metric. Success rate under wall-clock budget T (easy 30 s, middle 60 s, hard 100 s):

$$\text{SuccessRate} = \frac{1}{N}\big|\{i : \text{instance } i \text{ solved within } T\}\big|.$$

We also report failure reasons (`timeout` / `no_solution`) and success-vs-time curves. Fixed-budget success matches batch design use better than optimality claims we do not make.

Secondary metrics. Runtime on successes; on successes, cost_ratio = cost(sol)/cost($\mathcal{W}$) as a descriptive quality signal (*not* an optimality gap—$\mathcal{W}$ is constructive). Cost comparisons across algorithms use the common success set to limit selection bias.

We do not claim CBS optimality, uniqueness of $\mathcal{W}$, or cross-paper leaderboard status until the suite and scripts are frozen and released.

Algorithm 1 AxisRay: generate witness corridors

Require: domain size, radius r, groups $\{n_t^{(g)}\}$, waypoint budgets, gaps
Ensure: per-group corridors or **fail**

Occ $\leftarrow \emptyset$; WP$[g] \leftarrow \emptyset$ for all groups g
while some group needs more face terminals **do**
  for group g in round-robin order **do**
    if g has no corridor **then**
      sample face terminal s; grow seed chain with `init_waypoints_num`-2 interior LOS segments
      final LOS segment to a domain face; record waypoints
      on fail: rollback Occ / paths for this attempt
    else
      pick branch point p on g's tree with dist$(p, \text{WP}[g]) >$ `branch_gap`
      pick side-exit direction (not along the current edge axis)
      grow attach chain (`attach_waypoints_num`) to a new face
      on fail: rollback this attach
    end if
  end for
  if no progress after max attempts **then**
    return fail
  end if
end while
return corridors

References

[1] TODO: cite Additive Manufacturing for hydraulic valves / rotary direct-drive servo valves (replace with real references).

A Generator Details

This appendix records constructive generators used in Sec. 3. It is intended for reproducibility; the main text only needs the high-level behaviour.

A.1 AxisRay

AxisRay uses LOS scans with rollback on failed seed/attach attempts. Hard rules include: no entry into written path cells; Chebyshev gap `path_gap` except along the spine of the edge being left (requires `branch_gap`>`path_gap`); consecutive segments change axis; branch points stay farther than `branch_gap` from existing waypoints.

A.2 PlaneSlice

Every search is restricted to the active axis plane (2D); there is no fallback to full 3D A*. Same-group corridors off the plane are ignored during search; other groups on the plane are obstacles. Same-axis planes within a group obey a plane-gap.

Algorithm 2 PLANESLICE: generate witness corridors

Require: domain, r, subdomain N, plane gap, targets $\{n_t^{(g)}\}$
Ensure: per-group corridors or **fail**

build subdomain partition; Planes$[g] \leftarrow \emptyset$
while some group needs more terminals **do**
  for group g in round-robin order **do**
    if g empty **then**
      sample valid axis-plane P; place two face terminals s, t on P
      planar A* on P only; record path and P
    else
      sample plane type $\neq$ last plane; derive P from a cell of g
      multi-source starts $\leftarrow$ all cells of g lying on P
      pick new face goal on P; planar multi-source A* on P
      on success: append path, record P
    end if
  end for
end while
return corridors

Algorithm 3 PILLARFIELD: paint AABB obstacles

Require: instance with witness, $\rho^\star$, edge bounds, `dist_cap`

mark thickened witness as blocked; build Chebyshev distance field $d(\cdot)$
$V_0 \leftarrow$ count of free cells; $V_{\text{obs}} \leftarrow 0$
while $V_{\text{obs}}/V_0 < \rho^\star$ and progress **do**
  sample seed c with weight $\propto \min(d(c), \texttt{dist_cap})$
  expand AABB from c (greedy / isometric); reject if too thin
  among candidates, commit the largest-volume box into blocked
  update $d(\cdot)$ from newly blocked cells; V_{obs} += volume
end while
emit committed boxes as static obstacles

A.3 PillarField and CorridorRack

Let C be witness cells thickened by `corridor_margin` (and r). Eligible volume V_0 is the complement of C at paint start.

Algorithm 4 CorridorRack: complement of dilated witnesses

Require: instance with witness, dilate steps k

1: $F \leftarrow$ Chebyshev dilate of all witness cells by k
2: ObsCells $\leftarrow$ domain $\setminus F$
3: merge ObsCells into large AABBs; append to static obstacles
4: record derived $\rho \leftarrow V_{\text{obs}}/V_{\text{domain}}$